Университет ИТМО

Мегафакультет компьютерных технологий и управления

Факультет программной инженерии и компьютерной техники

Отчет
по лабораторной работе №3
«бизнес-логика программных систем»

Вариант 111

Выполнили:

Григорий Садовой

Байрамгулов Мунир

Преподаватель

Кривоносов Егор Дмитриевич

Санкт-Петербург, 2026 г.

Условие задания

Доработать приложение из лабораторной работы #2, реализовав в нём асинхронное выполнение задач с распределением бизнес-логики между несколькими вычислительными узлами и выполнением периодических операций с использованием планировщика задач, а также интеграцию с внешней информационной системой.

Требования к реализации асинхронной обработки:

1. Перед выполнением работы необходимо согласовать с преподавателем набор прецедентов, в реализации которых целесообразно использование асинхронного распределённого выполнения задач. Если таких прецедентов использования в имеющейся бизнес-процесса нет, нужно согласовать реализацию новых прецедентов, доработав таким образом модель бизнес-процесса из лабораторной работы #1.
2. Асинхронное выполнение задач должно использовать модель доставки «подписка».
3. В качестве провайдера сервиса асинхронного обмена сообщениями необходимо использовать сервис подписки на базе Apache Kafka + ZooKeeper.
4. Для отправки сообщений необходимо использовать Kafka Producer API.
5. Для получения сообщений необходимо использовать клиент Kafka на базе Spring Boot.

Требования к реализации распределённой обработки:

1. Обработка сообщений должна осуществляться на двух независимых друг от друга узлах сервера приложений.
2. Если логика сценария распределённой обработки предполагает транзакционность выполняемых операций, они должны быть включены в состав распределённой транзакции.

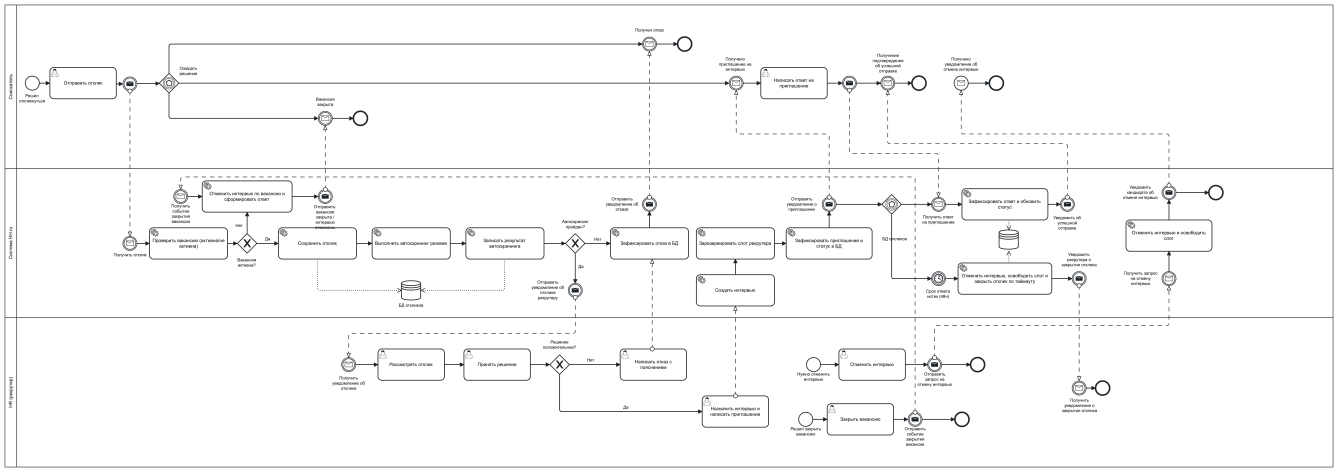
Требования к реализации запуска периодических задач по расписанию:

1. Согласовать с преподавателем прецедент или прецеденты, в рамках которых выглядит целесообразным использовать планировщик задач. Если такие прецеденты отсутствуют – согласовать с преподавателем новые и добавить их в модель автоматизируемого бизнес-процесса.
2. Реализовать утверждённые прецеденты с использованием планировщика задач Spring (@Scheduled).

Требования к интеграции с внешней Корпоративной Информационной Системой (EIS):

1. Корпоративная Информационная Система, с которой производится интеграция, а также её функциональные возможности выбираются на усмотрение преподавателя и согласуются с ним.
2. Взаимодействие с внешней Корпоративной Информационной Системой должно быть реализовано с помощью технологии JCA (Jakarta Connectors).

Модель потока управления и бизнес-процесса



Дорожки ответственности

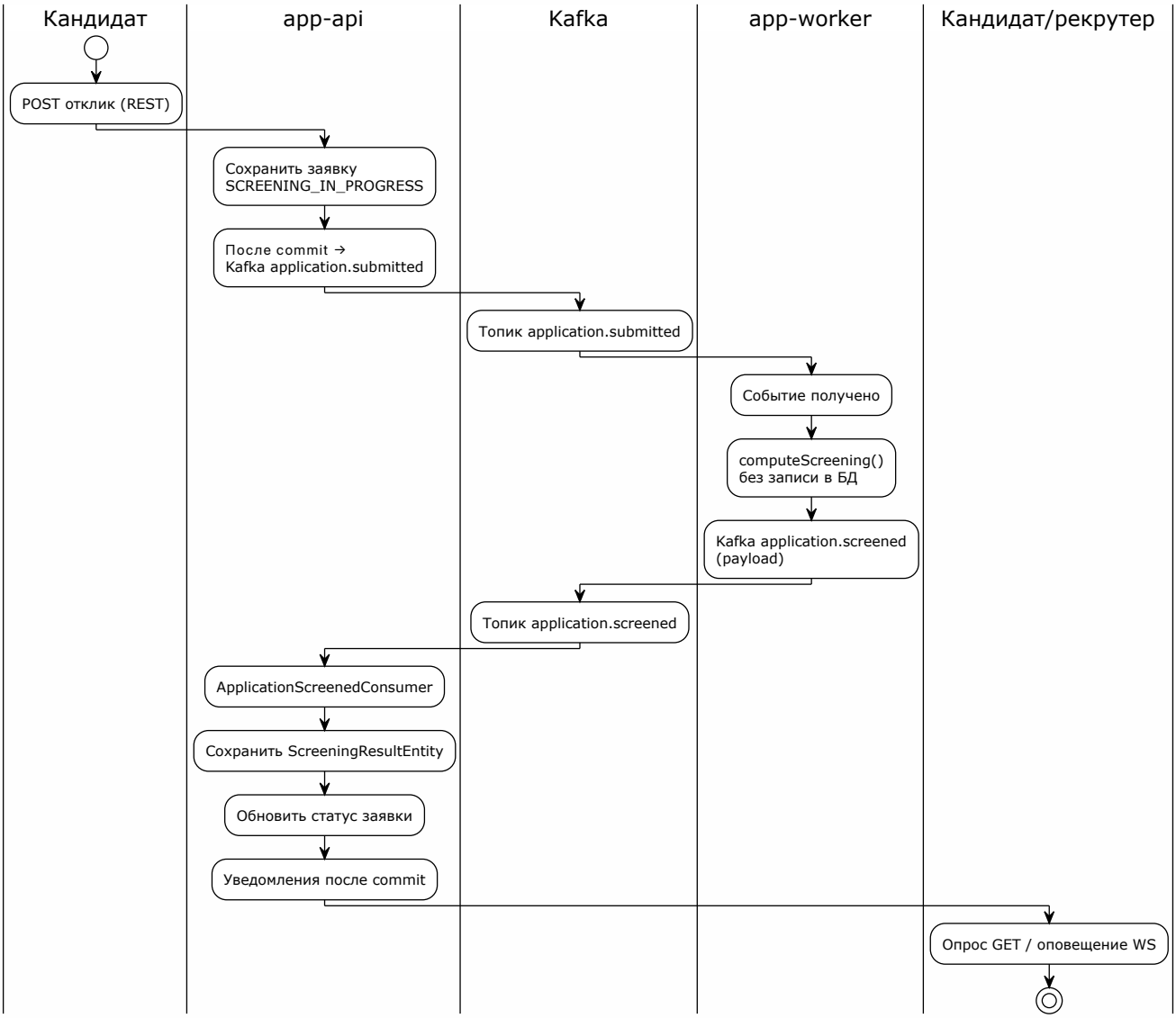


Диаграмма развёртывания

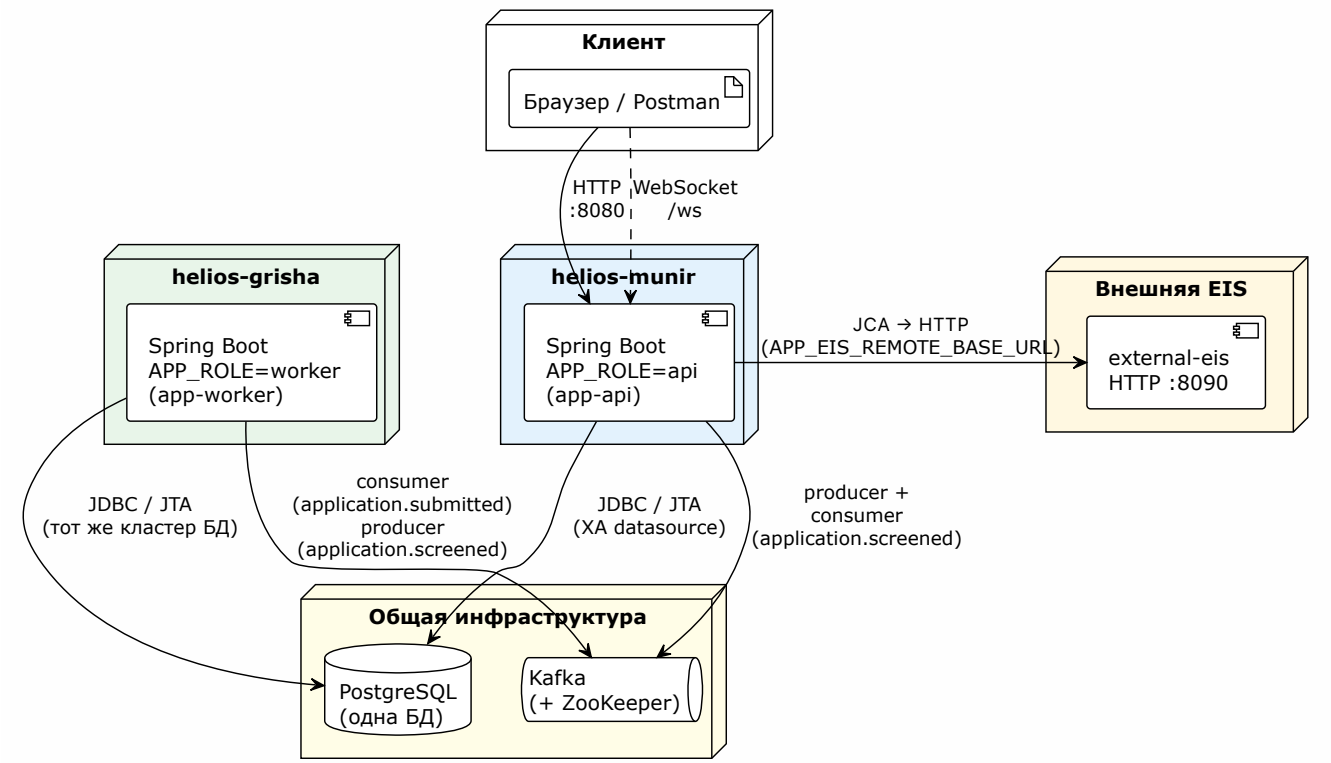
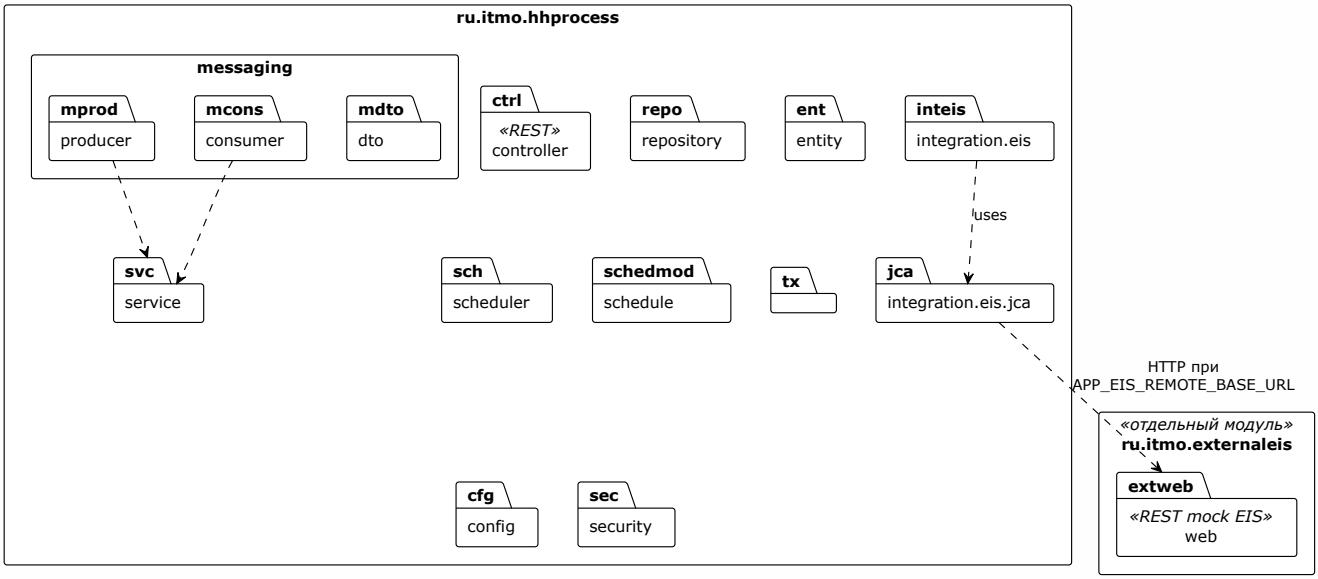


Рис. 2. Deployment Diagram: серверы `helios-munir` (api) и `helios-grisha` (worker), одна PostgreSQL, Kafka и JCA ⟶ HTTP к `external-eis` (PlantUML — `diagrams/deployment.puml`).

На production-хосте (GitHub Actions) дополнительно выкладывается JAR `external-eis` и скрипт `eisctl.sh` в каталог `~/apps/blps-eis`; API получает `APP_EIS_REMOTE_BASE_URL=http://127.0.0.1:8090`.

Диаграмма пакетов



Спецификация публичных интерфейсов (REST и WebSocket)

REST API

Метод	Путь	Назначение
POST	`/api/v1/auth/register/candidate`	Регистрация кандидата (доступ настраивается в `SecurityConfig`).
GET	`/api/v1/me`	Профиль текущего пользователя.
POST	`/api/v1/candidates/vacancies/\{vacancyId\}`	Подать заявку на вакансию; ответ: `APPLICATION_SUBMITTED`.
GET	`/api/v1/candidates/applications`	Список своих заявок.
GET	`/api/v1/candidates/applications/\{applicationId\}`	Заявка по id.
POST	`/api/v1/candidates/applications/\{applicationId\}/invitation-response`	Ответ на приглашение.
POST	`/api/v1/recruiters/vacancies`	Создать вакансию.
GET	`/api/v1/recruiters/vacancies`	Свои вакансии.
PATCH	`/api/v1/recruiters/vacancies/\{vacancyId\}/status`	Изменить статус вакансии.
POST	`/api/v1/recruiters/vacancies/\{vacancyId\}/close`	Закрыть вакансию (составная логика).
GET	`/api/v1/recruiters/applications`	Заявки по вакансиям рекрутера (фильтры query).
GET	`/api/v1/recruiters/applications/\{applicationId\}`	Заявка рекрутера по id.
POST	`/api/v1/recruiters/applications/\{applicationId\}/reject`	Отклонить заявку.
POST	`/api/v1/recruiters/applications/\{applicationId\}/invite`	Пригласить на интервью (JTA).
POST	`/api/v1/recruiters/interviews/\{interviewId\}/cancel`	Отменить интервью.
GET	`/api/v1/recruiters/schedule`	Недельное расписание рекрутера (`weekOffset`).
GET	`/api/v1/notifications`	Список уведомлений пользователя.
PATCH	`/api/v1/notifications/\{notificationId\}/read`	Отметить прочитанным.
POST	`/api/v1/admin/jobs/close-expired-invitations`	Принудительно закрыть просроченные приглашения.
POST	`/api/v1/admin/jobs/export-interviews`	Запуск экспорта интервью в EIS (JCA).
POST	`/api/v1/admin/debug/schedule-failure/\{enabled\}`	Debug: ошибка резерва слота (таблица `recruiter_schedule_slots`).

Таблица 1. Публичные HTTP-интерфейсы приложения `hh-process` (роль `api`)

Ответ при создании заявки (фрагмент JSON):

```JSON
{
  "application_id": "…",
  "status": "APPLICATION_SUBMITTED",
  "message": "Application submitted"
```

```
}
```

WebSocket (STOMP)

- Точка подключения: `/ws` (с поддержкой SockJS и без).
- Брокер: префиксы `/topic`, `/queue`; приложение — `/app`; пользовательские каналы — `/user`.
- Уведомления доставляются на `/user/queue/notifications` (см. `WebSocketNotificationService`).

Внешний сервис `external-eis`

Метод	Путь	Назначение
POST	`/api/v1/calendar/entries`	Создать запись календаря (экспорт интервью).
POST	`/api/v1/calendar/entries/\{interviewId\}/cancel`	Отмена записи.
GET	`/api/v1/calendar/entries/\{interviewId\}`	Получить запись.
GET	`/actuator/health`	Health (для оркестрации).

Таблица 2. HTTP API процесса `external-eis`

Периодические задачи

В системе присутствуют два scheduler use case на узле `api`:

- `TimeoutService.closeExpiredInvitations()` — закрытие просроченных приглашений;
- `InterviewExportScheduler.scheduleInterviewExport()` — выбор интервью и запуск экспорта в EIS (через `InterviewExportRequestService` и JCA после commit).

Обе задачи реализованы через Spring `@Scheduled`.

Выводы

В ходе ЛР3 исходный сценарий ЛР2 был доработан в соответствии с требованиями: асинхронность на Kafka, независимые узлы `api` и `worker` (в отчёте — серверы `helios-munir` и `helios-grisha`), идемпотентность потребителей, периодические задачи, транзакции JTA/Narayana в одной PostgreSQL и интеграция с корпоративной системой через JCA.

Основные результаты:

- **Screening** вынесен в асинхронную цепочку: `application.submitted` → worker (расчёт) → `application.screened` → api (фиксация в БД и уведомления).
- **Уведомления** доставляются с узла `api` напрямую в БД и по WebSocket, без отдельного Kafka-топика.
- **Экспорт в EIS** выполняется с узла `api` через JCA; при развёртывании добавлен отдельный HTTP-процесс `external-eis`, с которым адаптер общается по REST.
- Сохранена **модель безопасности** HTTP Basic и XML-пользователей; публичные интерфейсы задокументированы в отчёте и в OpenAPI (Swagger).

Практическая значимость работы — демонстрация сочетания синхронного REST API, асинхронной шины событий, транзакционной согласованности данных в нескольких ресурсах и стандартизированного подхода к интеграции с внешней системой (JCA + отдельный контур EIS).